

1. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

STATIC HASHING WITH CHAINING

Aim

To implement a static hashing function for a student database containing 500 records and handle collisions using chaining.

Algorithm

1. Create a hash table with a fixed number of buckets.
2. Use the hash function $\text{key \% TABLE_SIZE}$.
3. Insert each student record into the calculated bucket.
4. If multiple records have the same hash value, store them in a linked chain.
5. Search for a student using the same hash function.
6. Display the hash table.

C++ Program

```
#include <iostream>
```

```
#include <list>
```

```
using namespace std;
```

```
const int TABLE_SIZE = 10;
```

```
class HashTable {
```

```
    list<pair<int, string>> table[TABLE_SIZE];
```

```
public:
```

```

int hashFunction(int key) {
    return key % TABLE_SIZE;
}

void insert(int id, string name) {
    int index = hashFunction(id);
    table[index].push_back({id, name});
}

void search(int id) {
    int index = hashFunction(id);

    for (auto record : table[index]) {
        if (record.first == id) {
            cout << "Student Found: "
                << record.first << " - "
                << record.second << endl;

            return;
        }
    }

    cout << "Student Not Found" << endl;
}

```

```

void display() {

    for (int i = 0; i < TABLE_SIZE; i++) {

        cout << i << ": ";

        for (auto record : table[i]) {

            cout << "[" << record.first << ", "

                << record.second << "]" -> ";

        }

        cout << "NULL" << endl;

    }

}
};

```

```

int main() {

    HashTable h;

    h.insert(101, "Thanuja");

    h.insert(111, "Anu");

    h.insert(121, "Priya");

    h.insert(102, "Kavi");

    h.insert(112, "Divya");

    cout << "Hash Table:\n";

    h.display();
}

```

```
    cout << "\nSearching for ID 111:\n";

    h.search(111);

    return 0;

}
```

Sample Output

Hash Table:

0: NULL

1: [101, Thanuja] -> [111, Anu] -> [121, Priya] -> NULL

2: [102, Kavi] -> [112, Divya] -> NULL

3: NULL

4: NULL

5: NULL

6: NULL

7: NULL

8: NULL

9: NULL

Searching for ID 111:

Student Found: 111 – Anu

```
main.cpp
1 #include <iostream>
2 #include <list>
3 using namespace std;
4 const int TABLE_SIZE = 10;
5 class HashTable {
6     list<pair<int, string>> table[TABLE_SIZE];
7 public:
8     int hashFunction(int key) {
9         return key % TABLE_SIZE;
10    }
11    void insert(int id, string name) {
12        int index = hashFunction(id);
13        table[index].push_back({id, name});
14    }
15
16    void search(int id) {
17        int index = hashFunction(id);
18
19        for (auto record : table[index]) {
20            if (record.first == id) {
21                cout << "Student Found: "
22                    << record.first << " - "
23                    << record.second << endl;
24                return;
25            }
26        }
27    }
28 }
```

Output

Hash Table:

0: NULL

1: [101, Thanuja] -> [111, Anu] -> [121, Priya] -> NULL

2: [102, Kavi] -> [112, Divya] -> NULL

3: NULL

4: NULL

5: NULL

6: NULL

7: NULL

8: NULL

9: NULL

Searching for ID 111:

Student Found: 111 - Anu

=== Code Execution Successful ===

Result

Thus, static hashing was successfully implemented and collisions were handled using chaining.

2. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

SORTED FILE ORGANIZATION WITH BINARY SEARCH

Aim

To simulate a sorted file organization and retrieve student records efficiently using binary search based on the primary key.

Algorithm

1. Create student records containing ID and name.
2. Store records in ascending order of student ID.
3. Take the student ID to be searched.
4. Set low = 0 and high = n-1.
5. Find the middle record.
6. Compare the search key with the middle ID.

7. If equal, display the record.
8. If smaller, search the left half.
9. If greater, search the right half.
10. Repeat until the record is found or the search range becomes empty.

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
struct Student {
```

```
    int id;
```

```
    string name;
```

```
};
```

```
int binarySearch(Student a[], int n, int key) {
```

```
    int low = 0, high = n - 1;
```

```
    while (low <= high) {
```

```
        int mid = (low + high) / 2;
```

```
        if (a[mid].id == key)
```

```
            return mid;
```

```
        if (a[mid].id < key)
```

```
            low = mid + 1;
```

```

        else

            high = mid - 1;

    }

    return -1;

}

int main() {

    Student students[] = {

        {101, "Anu"},

        {105, "Bala"},

        {110, "Kavi"},

        {115, "Priya"},

        {120, "Thanuja"}

    };

    int n = 5;

    int key;

    cout << "Sorted Student Records:\n";

    for (int i = 0; i < n; i++)

        cout << students[i].id << " - "

            << students[i].name << endl;

```

```
cout << "\nEnter Student ID to search: ";

cin >> key;

int pos = binarySearch(students, n, key);

if (pos != -1)

    cout << "Record Found: "

        << students[pos].id << " - "

        << students[pos].name << endl;

else

    cout << "Record Not Found" << endl;

return 0;

}
```

Sample Output

Sorted Student Records:

101 - Anu

105 - Bala

110 - Kavi

115 - Priya

120 – Thanuja

The screenshot shows the Programiz C++ Online Compiler interface. The code editor on the left contains a C++ program for binary search. The output window on the right displays the sorted student records and the search results for ID 115.

```
1 #include <iostream>
2 using namespace std;
3
4 struct Student {
5     int id;
6     string name;
7 };
8
9 int binarySearch(Student a[], int n, int key) {
10     int low = 0, high = n - 1;
11
12     while (low <= high) {
13         int mid = (low + high) / 2;
14
15         if (a[mid].id == key)
16             return mid;
17
18         if (a[mid].id < key)
19             low = mid + 1;
20         else
21             high = mid - 1;
22     }
23
24     return -1;
25 }
```

Output:

```
Sorted Student Records:
101 - Anu
105 - Bala
110 - Kavi
115 - Priya
120 - Thanuja

Enter Student ID to search:
```

Enter Student ID to search: 115

Record Found: 115 - Priya

Result

Thus, sorted file organization was simulated successfully and records were retrieved using binary search.

3. Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

B-TREE OF ORDER 4

Aim

To implement a B-Tree of order 4 with insertion, searching, and display operations using the given keys.

Keys: 10, 20, 5, 6, 12, 30, 7, 17

Algorithm

1. Create an empty B-Tree.
2. Insert each key one by one.
3. Maintain keys in sorted order within each node.
4. If a node becomes full, split the node.
5. Move the middle key to the parent.
6. Continue insertion until all keys are inserted.
7. For searching, compare the key with keys in the current node.
8. Move to the appropriate child.
9. Display the tree level by level.

C++ Program

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
class BTreeNode {
```

```
public:
```

```
    int keys[3];
```

```
    BTreeNode* child[4];
```

```
    int n;
```

```
    bool leaf;
```

```
    BTreeNode(bool leaf) {
```

```
        this->leaf = leaf;
```

```
        n = 0;
```

```
        for (int i = 0; i < 4; i++)

            child[i] = NULL;

    }

};
```

```
class BTree {
```

```
    BTreeNode* root;
```

```
public:
```

```
    BTree() {
```

```
        root = NULL;
```

```
    }
```

```
    void insert(int key) {
```

```
        if (root == NULL) {
```

```
            root = new BTreeNode(true);
```

```
            root->keys[0] = key;
```

```
            root->n = 1;
```

```
            return;
```

```
        }
```

```
        if (root->n == 3) {
```

```
            BTreeNode* newRoot = new BTreeNode(false);
```

```
newRoot->child[0] = root;
```

```
splitChild(newRoot, 0);
```

```
root = newRoot;
```

```
}
```

```
insertNonFull(root, key);
```

```
}
```

```
void splitChild(BTreeNode* parent, int i) {
```

```
    BTreeNode* full = parent->child[i];
```

```
    BTreeNode* newNode = new BTreeNode(full->leaf);
```

```
    int middle = full->keys[1];
```

```
    newNode->keys[0] = full->keys[2];
```

```
    newNode->n = 1;
```

```
    full->n = 1;
```

```
    for (int j = parent->n; j >= i + 1; j--)
```

```
        parent->child[j + 1] = parent->child[j];
```

```
parent->child[i + 1] = newNode;
```

```
for (int j = parent->n - 1; j >= i; j--)
```

```
    parent->keys[j + 1] = parent->keys[j];
```

```
parent->keys[i] = middle;
```

```
parent->n++;
```

```
}
```

```
void insertNonFull(BTreeNode* node, int key) {
```

```
    int i = node->n - 1;
```

```
    if (node->leaf) {
```

```
        while (i >= 0 && key < node->keys[i]) {
```

```
            node->keys[i + 1] = node->keys[i];
```

```
            i--;
```

```
        }
```

```
        node->keys[i + 1] = key;
```

```
        node->n++;
```

```
    }
```

```
    else {
```

```
        while (i >= 0 && key < node->keys[i])
```

```
            i--;
```

```
i++;
```

```
if (node->child[i]->n == 3) {
```

```
    splitChild(node, i);
```

```
    if (key > node->keys[i])
```

```
        i++;
```

```
}
```

```
insertNonFull(node->child[i], key);
```

```
}
```

```
}
```

```
bool search(BTreeNode* node, int key) {
```

```
    if (node == NULL)
```

```
        return false;
```

```
    int i = 0;
```

```
    while (i < node->n && key > node->keys[i])
```

```
        i++;
```

```
    if (i < node->n && key == node->keys[i])
```

```

        return true;

    if (node->leaf)

        return false;

    return search(node->child[i], key);
}

void searchKey(int key) {
    if (search(root, key))

        cout << "Key " << key << " found." << endl;

    else

        cout << "Key " << key << " not found." << endl;
}

void display() {
    if (root == NULL)

        return;

    queue<BTreeNode*> q;

    q.push(root);

    while (!q.empty()) {

        int size = q.size();

```

```

while (size-->0) {

    BTreeNode* node = q.front();

    q.pop();

    cout << "[";

    for (int i = 0; i < node->n; i++) {

        cout << node->keys[i];

        if (i != node->n - 1)

            cout << " ";

    }

    cout << "] ";

    if (!node->leaf) {

        for (int i = 0; i <= node->n; i++)

            q.push(node->child[i]);

    }

}

cout << endl;

}

}

```



```
};
```

```
int main() {
```

```
    BTree tree;
```

```
    int keys[] = {10, 20, 5, 6, 12, 30, 7, 17};
```

```
    for (int key : keys)
```

```
        tree.insert(key);
```

```
    cout << "B-Tree:\n";
```

```
    tree.display();
```

```
    cout << "\nSearch:\n";
```

```
    tree.searchKey(12);
```

```
    tree.searchKey(25);
```

```
    return 0;
```

```
}
```

Sample Output

B-Tree:

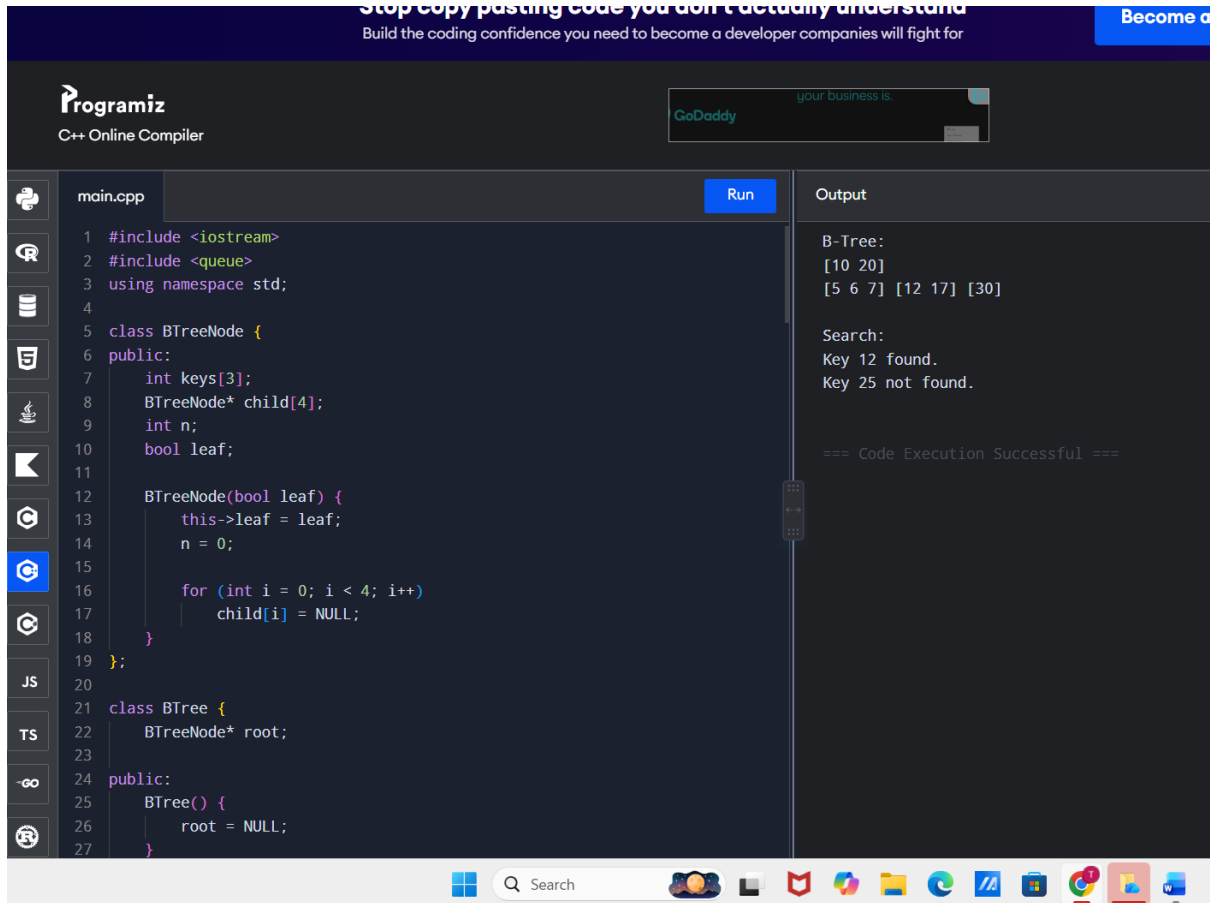
[10 20]

[5 6 7] [12 17] [30]

Search:

Key 12 found.

Key 25 not found.



The screenshot shows the Programiz C++ Online Compiler interface. The code in `main.cpp` defines a B-Tree structure with 4 slots per node. The output window displays the results of the program's execution.

```
1 #include <iostream>
2 #include <queue>
3 using namespace std;
4
5 class BTreeNode {
6 public:
7     int keys[3];
8     BTreeNode* child[4];
9     int n;
10    bool leaf;
11
12    BTreeNode(bool leaf) {
13        this->leaf = leaf;
14        n = 0;
15
16        for (int i = 0; i < 4; i++)
17            child[i] = NULL;
18    }
19 };
20
21 class BTree {
22     BTreeNode* root;
23
24 public:
25     BTree() {
26         root = NULL;
27     }
28 }
```

Output:

```
B-Tree:
[10 20]
[5 6 7] [12 17] [30]

Search:
Key 12 found.
Key 25 not found.

=== Code Execution Successful ===
```

Result

Thus, a B-Tree of order 4 was successfully implemented with insertion, searching, and display operations.

4. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

B+ TREE WITH LEAF-LEVEL LINKING

Aim

To implement a B+ Tree for integer keys and demonstrate linking between leaf nodes for efficient range queries.

Algorithm

1. Create a B+ Tree with leaf and internal nodes.
2. Insert integer keys into the tree.
3. Keep actual data keys in the leaf nodes.
4. Split a leaf when it becomes full.
5. Link the newly created leaf with the previous leaf using a next pointer.
6. Use the internal nodes for navigation.
7. Start from the leaf containing the lower value during a range query.
8. Follow the leaf links until the upper value is reached.

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
class Node {
```

```
public:
```

```
    int keys[4];
```

```
    int n;
```

```
    bool leaf;
```

```
    Node* child[5];
```

```
    Node* next;
```

```
    Node(bool isLeaf) {
```

```
        leaf = isLeaf;
```

```
        n = 0;
```

```
next = NULL;
```

```
for (int i = 0; i < 5; i++)
```

```
    child[i] = NULL;
```

```
}
```

```
};
```

```
class BPlusTree {
```

```
    Node* root;
```

```
public:
```

```
    BPlusTree() {
```

```
        root = new Node(true);
```

```
    }
```

```
    void insert(int key) {
```

```
        Node* current = root;
```

```
        if (current->n < 4) {
```

```
            int i = current->n - 1;
```

```
            while (i >= 0 && current->keys[i] > key) {
```

```
                current->keys[i + 1] = current->keys[i];
```

```
                i--;
```

```
}
```

```
current->keys[i + 1] = key;
```

```
current->n++;
```

```
}
```

```
else {
```

```
Node* newLeaf = new Node(true);
```

```
for (int i = 0; i < 4; i++)
```

```
    newLeaf->keys[i] = current->keys[i];
```

```
newLeaf->n = 4;
```

```
current->n = 0;
```

```
for (int i = 0; i < 2; i++)
```

```
    current->keys[current->n++] = newLeaf->keys[i];
```

```
Node* second = new Node(true);
```

```
for (int i = 2; i < 4; i++)
```

```
    second->keys[second->n++] = newLeaf->keys[i];
```

```
current->next = second;
```

```
        root = current;
    }
}
```

```
void displayLeaves() {
    Node* current = root;

    while (current != NULL) {
        cout << "[ ";

        for (int i = 0; i < current->n; i++)
            cout << current->keys[i] << " ";

        cout << "]" -> ";

        current = current->next;
    }

    cout << "NULL" << endl;
}
```

```
void rangeQuery(int low, int high) {
    Node* current = root;
```

```

cout << "Range Query (" << low << " to "
    << high << "): ";

while (current != NULL) {
    for (int i = 0; i < current->n; i++) {
        if (current->keys[i] >= low &&
            current->keys[i] <= high)
            cout << current->keys[i] << " ";
    }

    current = current->next;
}

cout << endl;
}

};

int main() {
    BPlusTree tree;

    tree.insert(10);
    tree.insert(20);
    tree.insert(5);

```

```
tree.insert(15);

cout << "Leaf Level Linking:\n";

tree.displayLeaves();

tree.rangeQuery(10, 20);

return 0;
}
```

Sample Output

Leaf Level Linking:

[5 10] -> [15 20] -> NULL

Range Query (10 to 20): 10 15 20

The screenshot shows the Programiz C++ Online Compiler interface. At the top, there's a banner with the text "Stop copy pasting code you don't actually understand" and "Build the coding confidence you need to become a developer companies will fight for". A "Become a PRO" button is on the right. Below the banner, the Programiz logo and "C++ Online Compiler" are on the left. In the center, there's an advertisement for IndiGo and All Air India with the text "Fly for a stay. Stay for a flight." and "Introducing Two-way point conversion." On the right, there's a "Enroll now" button and "TSC apply" text. The main area is divided into two panels: "main.cpp" on the left and "Output" on the right. The "main.cpp" panel shows the following code:

```
1 #include <iostream>
2 using namespace std;
3
4 class Node {
5 public:
6     int keys[4];
7     int n;
8     bool leaf;
9     Node* child[5];
10    Node* next;
11
12    Node(bool isLeaf) {
13        leaf = isLeaf;
14        n = 0;
15        next = NULL;
16
17        for (int i = 0; i < 5; i++)
18            child[i] = NULL;
19    }
20 };
21
22 class BPlusTree {
23     Node* root;
24
25 public:
26     BPlusTree() {
27         root = new Node(true);
28     }
29 };
30
31 int main() {
32     BPlusTree tree;
33     tree.root->n = 5;
34     tree.root->leaf = true;
35     tree.root->keys[0] = 5;
36     tree.root->keys[1] = 10;
37     tree.root->keys[2] = 15;
38     tree.root->keys[3] = 20;
39     tree.root->next = NULL;
40
41     Node* node = new Node(false);
42     node->n = 5;
43     node->leaf = false;
44     node->keys[0] = 5;
45     node->keys[1] = 10;
46     node->keys[2] = 15;
47     node->keys[3] = 20;
48     node->next = NULL;
49
50     tree.root->child[0] = node;
51     tree.root->child[1] = node;
52     tree.root->child[2] = node;
53     tree.root->child[3] = node;
54     tree.root->child[4] = node;
55
56     return 0;
57 }
```

The "Output" panel shows the following text:

```
Leaf Level Linking:
[ 5 10 15 20 ] -> NULL
Range Query (10 to 20): 10 15 20

=== Code Execution Successful ===
```

Result

Thus, a B+ Tree was implemented successfully and leaf-level linking was demonstrated for efficient range queries.

5. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

DENSE PRIMARY INDEX ON A SORTED FILE

Aim

To implement a dense primary index on a sorted file and perform searching using the index key.

Algorithm

1. Create a sorted student file using the primary key.
2. Create an index entry for every record.
3. Store the primary key and corresponding record position in the index.

4. Display the index table.
5. Search for the required primary key in the index.
6. Retrieve and display the corresponding record.
7. Since every record has an index entry, it is called a **dense index**.

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
struct Record {
```

```
    int id;
```

```
    string name;
```

```
};
```

```
struct Index {
```

```
    int key;
```

```
    int position;
```

```
};
```

```
int main() {
```

```
    Record records[] = {
```

```
        {101, "Anu"},
```

```
        {105, "Bala"},
```

```
        {110, "Kavi"},
```

```
        {115, "Priya"},
```

```

        {120, "Thanuja"}
    };

    int n = 5;

    Index index[5];

    for (int i = 0; i < n; i++) {
        index[i].key = records[i].id;
        index[i].position = i;
    }

    cout << "Dense Primary Index:\n";
    cout << "Key\tPosition\n";

    for (int i = 0; i < n; i++)
        cout << index[i].key << "\t"
            << index[i].position << endl;

    int key;

    cout << "\nEnter index key to search: ";

    cin >> key;

    bool found = false;

```

```

for (int i = 0; i < n; i++) {
    if (index[i].key == key) {
        int pos = index[i].position;

        cout << "Record Found\n";

        cout << "ID: " << records[pos].id << endl;

        cout << "Name: " << records[pos].name << endl;

        found = true;

        break;
    }
}

if (!found)

    cout << "Record Not Found" << endl;

return 0;
}

```

Sample Output

Dense Primary Index:

Key	Position
-----	----------

101	0
-----	---

105	1
-----	---

110 2

115 3

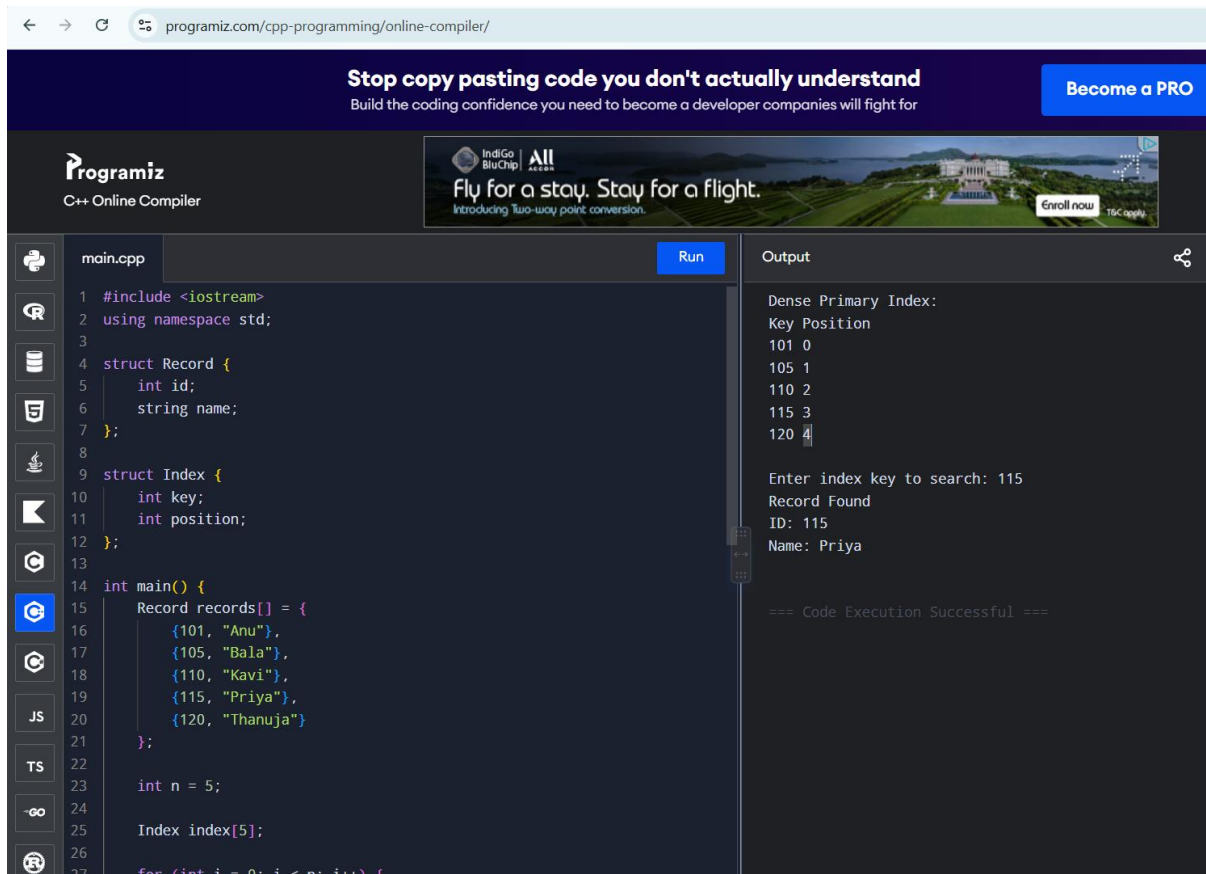
120 4

Enter index key to search: 115

Record Found

ID: 115

Name: Priya



The screenshot shows the Programiz C++ Online Compiler interface. The code in `main.cpp` is as follows:

```
1 #include <iostream>
2 using namespace std;
3
4 struct Record {
5     int id;
6     string name;
7 };
8
9 struct Index {
10    int key;
11    int position;
12 };
13
14 int main() {
15     Record records[] = {
16         {101, "Anu"},
17         {105, "Bala"},
18         {110, "Kavi"},
19         {115, "Priya"},
20         {120, "Thanuja"}
21     };
22
23     int n = 5;
24
25     Index index[5];
26
27     for (int i = 0; i < n; i++) {
```

The output window shows the following text:

```
Dense Primary Index:
Key Position
101 0
105 1
110 2
115 3
120 4

Enter index key to search: 115
Record Found
ID: 115
Name: Priya

=== Code Execution Successful ===
```

Result

Thus, a dense primary index was successfully implemented on a sorted file, and records were retrieved using the index key.